

OdooSeek 项目愿景文档

用现代 Web 技术重新定义 Odoo 的前端体验

一、项目命名

OdooSeek — **odoo** + **seek**，意为"探寻 Odoo 的最佳可能"。
前端代号 **oweb** — **odoo** + **web**，代表新生的 Odoo Web 客户端。
服务端代号 **odoo-web-server** — Rust 构建的中间层网关，连接前端与 Odoo。

对接目标

项目	详情
Odoo 版本	Odoo 19 Community Edition（社区版）
Odoo 源码	~/EA/odoo
API 风格	JSON-RPC (Odoo 原生协议)，无 Odoo Enterprise 依赖
协议	POST /jsonrpc 标准端点，Session Cookie 认证

二、缘起：为什么要做这个项目？

Odoo 前端的现实困境

Odoo 是全球领先的开源 ERP 系统，但其前端技术栈长期滞后于现代 Web 开发潮流：

现状	痛点
OWL (Odoo Web Library)	自研框架，生态孤立，社区资源匮乏
服务端渲染 (QWeb)	前后端强耦合，无法独立部署和迭代
jQuery 遗留代码	大量旧代码混杂，维护成本高
模块加载机制	基于 require.js 的 bundle 机制，不支持现代 Tree Shaking、Code Splitting
开发体验	无 HMR、构建缓慢、调试困难
人才壁垒	开发者需同时掌握 Python/Odoo + OWL，招聘困难

我们的回应

OdooSeek 的命题：

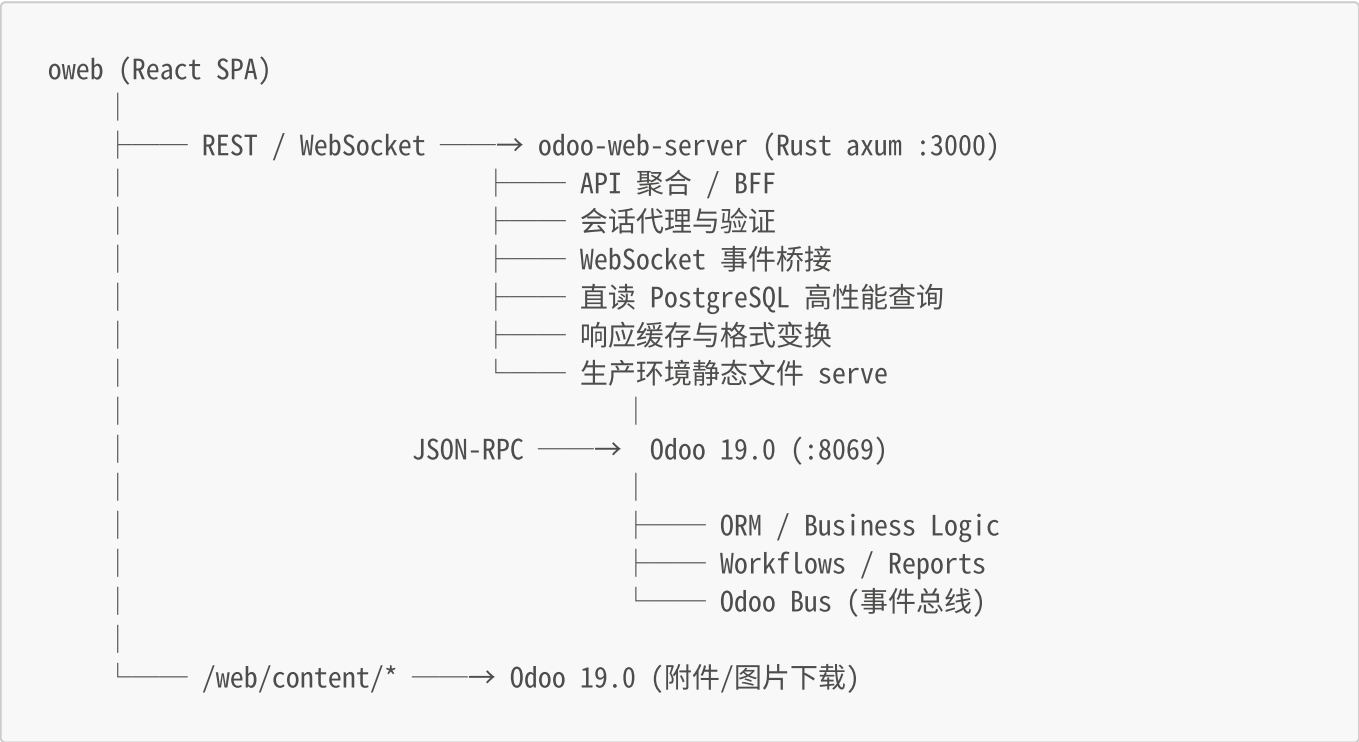
Odoo 的声明式元数据架构是它最宝贵的资产。我们不需要重写 CRM、会计引擎、工作流——我们需要的是：保留这一切，但给用户一个世界级的前端体验。

答案就是 **oweb** — 一个基于 React + TypeScript + Bun + TanStack/Router + Rust BFF 的 Odoo 前端项目，它：

- 保留 Odoo 的元数据驱动内核（`ir.*` 表、XML 视图声明、声明式权限）
- 保留 Odoo 的业务逻辑（Python ORM、工作流引擎、报表、自动化）
- 替换 Odoo 的前端渲染层（OWL/QWeb → React SPA）
- 替换 Odoo 的通信层（Python WSGI 直连 → Rust BFF 网关）

服务端架构：两段式后端

本项目采用 **前后端分离 + 两段式服务端** 架构，区别于传统 Odoo 单体部署：



组件	技术栈	职责
oweb	React + TypeScript + Vite + TanStack	用户界面，纯静态 SPA
odoo-web-server	Rust + axum + tokio + reqwest	API 网关/BFF，前后端之间的智能中间层
Odoo 19 CE	Python (源码 ~/EA/odoo) + PostgreSQL 16	业务逻辑核心，ORM，工作流引擎

架构哲学：延续 Odoo 的灵魂

Odoo 本质上是一个 **元数据解释器**。它的核心哲学是：

数据定义程序，而非程序定义数据。 — 模型、视图、菜单、权限、自动化流程，全部以声明式元数据的形式存储在数据库中，而非硬编码在源代码中。

具体表现：

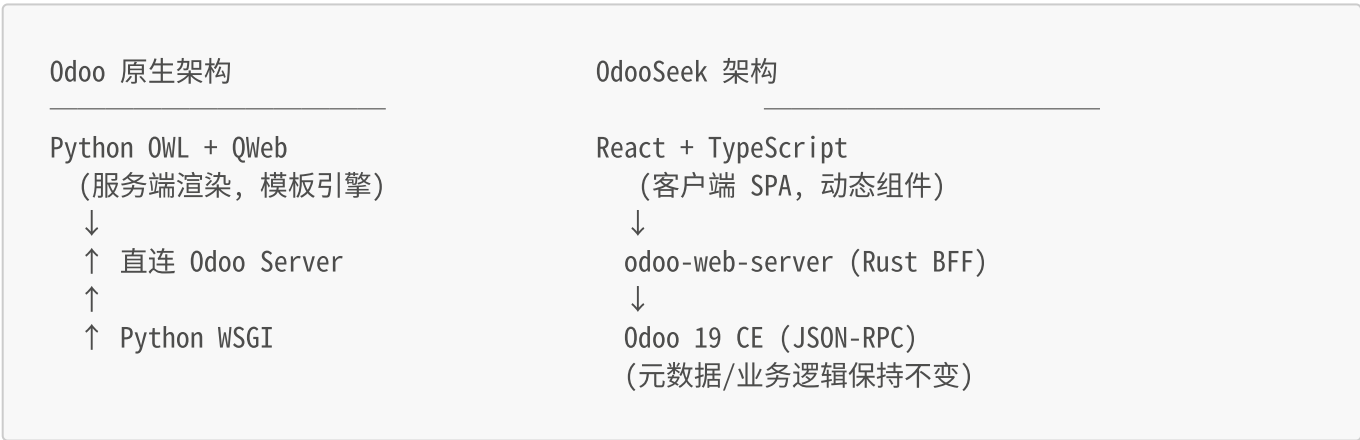
Odoo 元数据表	存储内容	运行时行为
-----------	------	-------

Odoo 元数据表	存储内容	运行时行为
<code>ir.model / ir.model.fields</code>	模型定义与字段属性	向数据库插入一条记录，即可动态新增字段——无需修改代码
<code>ir.ui.view</code>	表单、列表、看板的 XML 结构	前端请求时从数据库读取 XML，解析、继承、合并后渲染
<code>ir.ui.menu / ir.actions.*</code>	菜单项与动作定义	导航结构由数据库记录驱动，前端动态构建菜单
<code>ir.rule</code>	记录级权限规则	<code>[('company_id','=', user.company_id)]</code> 声明式过滤，无需 SQL
<code>ir.cron / base.automation</code>	定时任务与自动化	声明触发条件与执行动作，无循环或条件分支

OdooSeek 完整保留这套元数据驱动架构：

- 模型定义 — 仍由 Odoo ORM 管理，oweb 通过 JSON-RPC 查询 `ir.model.fields` 获取字段元数据
- 视图定义 — 仍存储在 `ir.ui.view` 表中，oweb 动态获取并解析 XML 视图结构
- 菜单与权限 — 仍由 Odoo 内置模块管理，oweb 通过 API 拉取后渲染
- 业务逻辑 — 全部保留在 Odoo 服务端，不迁移、不重写

我们改变的，只是前端渲染层和通信中间件：



"我们不是在逃离 Odoo，而是在拓宽它的边界。" — OdooSeek 保留了 Odoo 元数据驱动的设计灵魂，用现代 Web 技术赋予它一个全新的前端表达。

三、使命宣言

让 Odoo 拥有世界级的前端体验，降低 ERP 定制化的技术与人才门槛。

我们将：

- 解耦前后端 — 前端作为独立 SPA，通过 Rust BFF 与 Odoo JSON-RPC API 通信
- 拥抱主流生态 — 使用 React、TypeScript、TanStack（前端）+ Rust/axum（中间层）
- 追求极致 DX — Bun 极速构建、Vite HMR、TypeScript 全覆盖、Rust 零成本抽象
- 开放与包容 — 任何前端/后端开发者都能参与贡献，无需了解 Odoo 全套技术栈

四、技术选型

前端技术栈（oweb）

oweb 前端	
运行时	Bun (JavaScript/TypeScript 运行时)
UI 框架	React 19
类型系统	TypeScript 6 (strict mode)
路由	TanStack Router (类型安全路由)
状态管理	TanStack Query (服务端状态) Zustand (客户端状态)
构建工具	Vite 8
样式方案	Tailwind CSS 4
测试	Vitest + React Testing Library
API 通信	REST / WebSocket (到 Rust BFF)

中间层技术栈（odoo-web-server）

odoo-web-server (Rust)	
异步运行时	Tokio
HTTP 框架	Axum
中间件	Tower HTTP (CORS, 静态文件, 日志)
WebSocket	Axum + tokio::sync::broadcast
数据库直连	sqlx / tokio-postgres
Odoo RPC	Reqwest (HTTP JSON-RPC 客户端)
Session 管理	Cookie 透传 + Odoo session 验证
缓存	moka / dashmap (内存缓存)
序列化	Serde JSON

选型理由

Bun → 替代 Node.js

维度	Node.js	Bun
包管理	npm/yarn/pnpm	原生内置，快 25x

维度	Node.js	Bun
启动速度	~200ms	~30ms
TypeScript	需 ts-node/tsx	原生支持
测试	需 Jest/Vitest	内置 <code>bun test</code>
兼容性	Node.js API	完全兼容

选择 Bun 作为运行时的核心理由：**极致的开发体验**。从 `bun install` 到 `bun dev` 到 `bun test`，一条命令无需任何配置文件。

React 19 → UI 框架

- 全球最大的前端生态，人才池最广
- **React Server Components** 虽未直接使用，但 React 19 的并发特性为未来扩展留足空间
- 与 TanStack 生态完美配合

TanStack Router → 路由

- **类型安全是第一公民**：URL 参数、Search Params、Route Context 全程类型推断
- 支持嵌套路由、Layout Routes、路由级数据预加载（Loader）
- 比 React Router v6 更现代的设计哲学，与 TanStack Query 深度集成

TanStack Query → 服务端状态

- 将 Odoo 的 `search_read`、`read`、`write` 等 JSON-RPC 调用封装为声明式 Query/Mutation
- 内置缓存、后台刷新、乐观更新、分页/无限滚动
- 消除前端手工管理 `loading/error/data` 状态的样板代码

Vite → 构建工具

- 开发环境下秒级 HMR（热模块替换）
- 生产环境下基于 Rollup 的高效构建
- 与 Bun 配合使用极致加速

odoo-web-server → BFF 中间层

选择 Rust + Axum 构建中间层的核心理由：

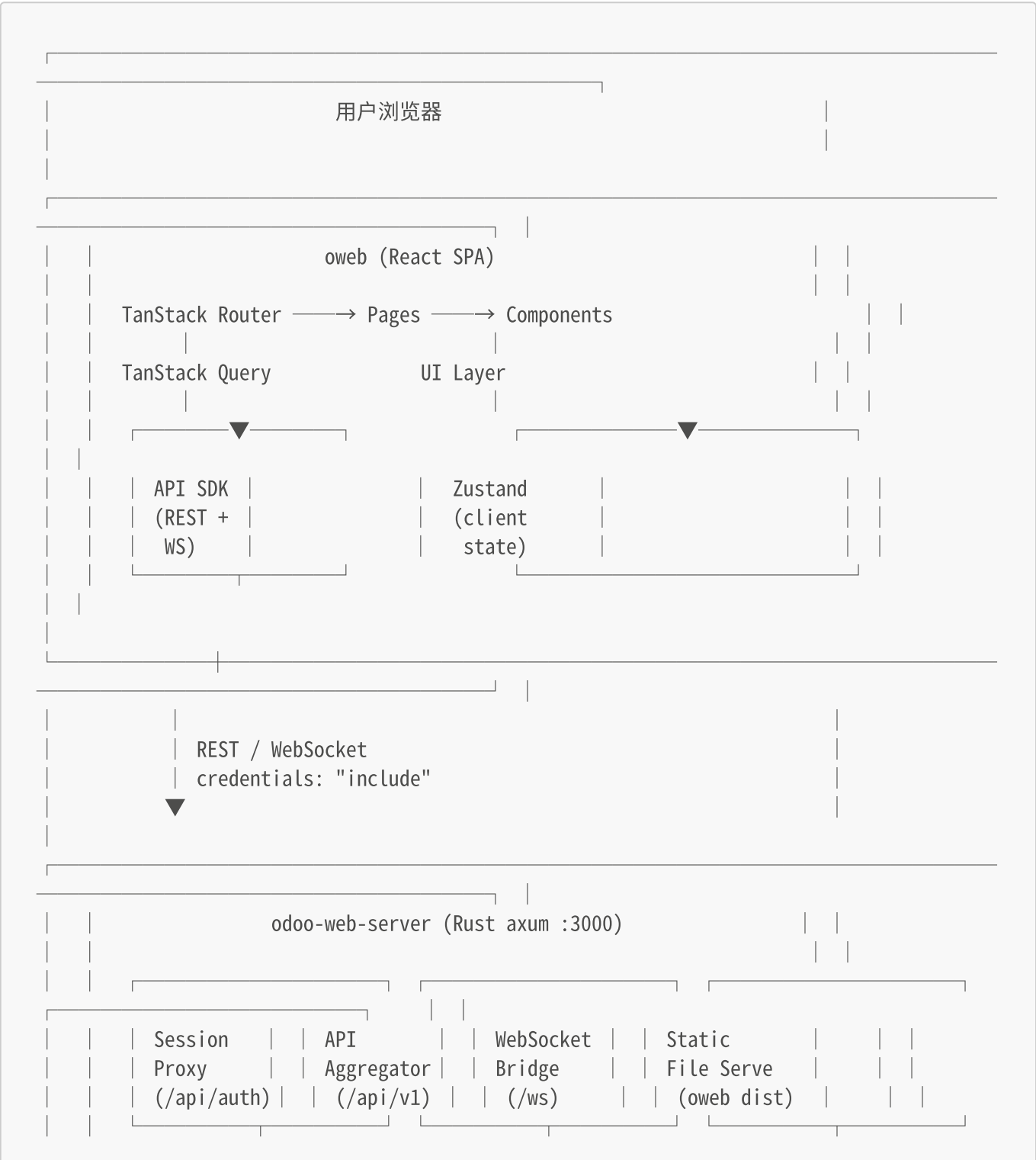
职责	说明
API 聚合	将前端多次 Odoo JSON-RPC 调用合并为单次 REST 请求，减少网络往返
会话代理	前端请求 → Rust 验证 Odoo session → 转发到 Odoo，前端无需直连 Odoo
WebSocket 桥接	接收 Odoo Bus 事件，转为标准 WebSocket 推送给 oweb
高性能直读	对仪表盘统计等性能敏感查询，Rust 直连 PostgreSQL，绕过 Odoo ORM
响应缓存	菜单树、模型元数据、用户信息等低变更数据缓存到内存，加速渲染

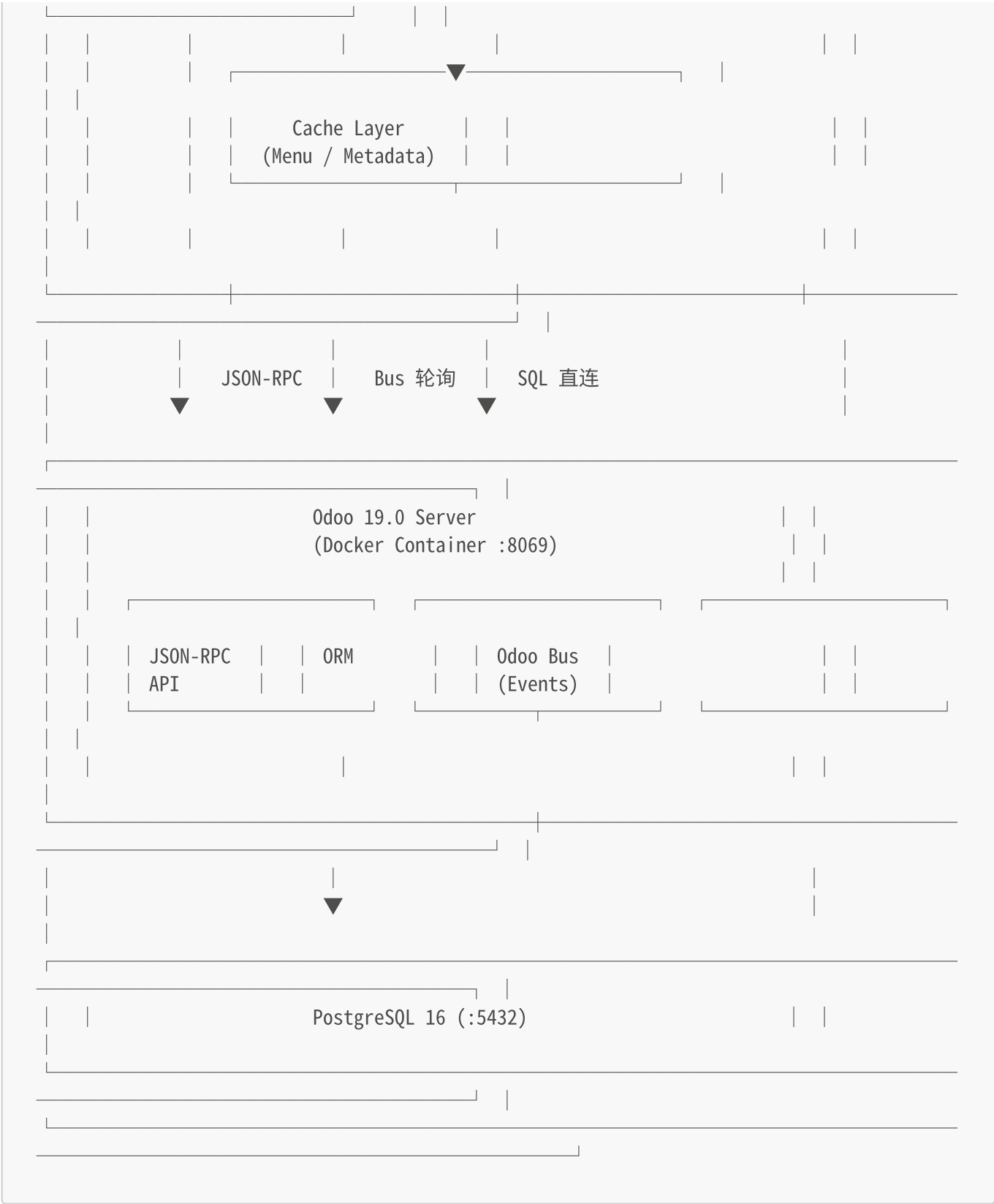
职责	说明
格式变换	将 Odoo 的 <code>[id, name]</code> 数组格式转为前端友好的 <code>{id, name}</code> JSON
类型同步	Rust struct → Serde JSON ↔ TypeScript interface，编译期保证契约一致

要点：odoo-web-server 是 **增强层而非替代层** — Odoo 仍然是业务核心，Rust 负责"让前端更高效地与 Odoo 对话"。

五、架构设计

整体架构





核心设计原则

1. 前后端彻底解耦

- oweb 是**纯静态 SPA**，构建产物可以部署到任何 Web 服务器（Nginx、CDN、S3）
- 与 Odoo 通信仅通过标准 HTTP 协议（JSON-RPC），不依赖 Odoo 模板引擎或 QWeb
- 可以连接任意版本的 Odoo（14/15/16/17/18/19），只需实现对应的 JSON-RPC Adapter

2. 类型安全贯穿全栈

Odoo Model (Python)		oweb (TypeScript)
res.partner		PartnerModel
name: Char	————>	name: string
email: Char	————>	email: string
phone: Char	————>	phone: string
company_id: M2O	————>	company_id: [number, string]

- 为每个 Odoo Model 定义 TypeScript 类型接口
- TanStack Router 的 Search Params 类型化，防止 URL 参数拼写错误
- 从 JSON-RPC 响应中自动推断类型，IDE 智能提示覆盖所有 API 调用

项目结构

odooseek/	
├── apps/	
│ ├── oweb/	# 前端 SPA (React + TypeScript + Vite)
│ │ ├── src/	
│ │ │ ├── routes/	# TanStack Router 页面
│ │ │ ├── components/	# 共享组件
│ │ │ ├── lib/	# API SDK + 工具库
│ │ │ └── themes/	# 运行时主题引擎
│ │ └── dist/	# Vite 构建产物
├── crates/	# Rust Workspace
│ ├── odoo-web-server/	# axum HTTP/WebSocket 服务端 (BFF)
│ │ └── src/main.rs	
│ ├── odoo-core/	# 共享类型: Session / Model / JSON-RPC
│ │ └── src/lib.rs	
├── docker/	
│ ├── docker-compose.yml	# 统一编排 (oweb + Rust + Odoo + PG)
│ ├── nginx/	# 过渡期反向代理 (Rust server 就绪后可移除)
│ └── 19.0/	# Odoo 19.0 镜像构建
└── docs/	
└── VISION.md	# 本文档

前端源码树 (apps/oweb/src/) :

├── routes/	# 页面路由 (TanStack Router)
│ ├── __root.tsx	# 根布局 (Navbar + Outlet)
│ ├── home.tsx	# 首页
│ ├── login.tsx	# Odoo 登录
│ ├── dashboard.tsx	# 会话仪表盘
│ └── settings.tsx	# 设置
├── components/	# 共享组件
└── ErrorBoundary.tsx	


```
(规划中：)
```

```
|—— modules/                # 业务模块（按 Odoo App 划分）
|   |—— sale/                # 销售模块
|   |—— crm/                 # CRM 模块
|   |—— ...
|—— router/
|   |—— guards.ts            # 路由守卫（认证、权限）
```

```
bun dev # 启动开发服务器，HMR 秒级更新
bun test --watch # 监听模式运行测试
bun run build # 生产构建，产物输出到 dist/
bun lint # Biome 格式化和 Lint
bun typecheck # TypeScript 类型检查
```

第一阶段：核心基建 (MVP)

任务	产出
Odoo Bus 桥接	Rust 轮询 Odoo Bus → WebSocket 推送到 oweb
Docker 编排	docker-compose.yml 整合 oweb + Rust + Odoo + PostgreSQL

第二阶段：功能模块

模块	覆盖范围
销售 (Sale)	报价单、销售订单、产品管理
CRM	商机管道、活动管理、客户 360
库存 (Inventory)	库存盘点、出入库、批次追踪
会计 (Accounting)	发票、凭证、科目余额表

第三阶段：体验增强

- PWA 离线支持
- 键盘快捷导航（已实现基础版本）
- 实时通知（Odoo Bus + WebSocket）
- 国际化（i18n）
- 通用列表/表单 Odoo View 渲染引擎

七、为什么这个项目值得做？

对 Odoo 生态的价值

1. **降低 Odoo 前端定制的技术门槛** — React/TypeScript 开发者无需学习 OWL 即可贡献
2. **提升 Odoo 的集成能力** — 标准 SPA 架构易于嵌入其他系统或被其他系统嵌入
3. **加速 Odoo 的现代化** — 社区的创新可以反哺 Odoo 官方前端演进

对开发者的价值

1. **技能可迁移** — 项目中获得的 React/TypeScript/TanStack 经验可应用于任何现代 Web 项目
2. **参与开源 ERP** — 以前端角度参与 ERP 开发，打破"ERP 开发必须懂后端"的刻板印象
3. **高质量的代码实践** — 项目坚持类型安全、自动化测试、代码审查，是学习工程最佳实践的绝佳场域

对企业的价值

1. **快速定制** — 成熟的 React 生态意味着成百上千可直接复用的 UI 组件
2. **降低运维成本** — 前后端独立部署，前端可以部署到 CDN
3. **人才易得** — 招聘 React 开发者远比招聘 Odoo OWL 开发者容易

八、协作与贡献

项目原则

- **代码即文档** — 自解释的变量和函数命名，超过 3 行的逻辑必有注释
- **类型即边界** — 所有跨模块接口必须有 TypeScript 类型定义
- **组件即故事** — 每个共享组件附带 Storybook 文档
- **测试即信心** — 核心逻辑覆盖率 > 80%

当前状态（2026-05）

☐ Docker 基础设施就绪	(odoo:19.0 镜像 + PostgreSQL 16 + Nginx)
☐ oweb 项目脚手架	(bun + Vite 8 + React 19 + TypeScript 6 + TanStack Router)
☐ Odoo RPC SDK	(lib/api.ts: authenticate / searchRead / getSession / destroySession)
☐ 核心路由 + 懒加载	(4 路由 + 3 条 lazy load, 主包 100KB gzip)
☐ 运行时主题引擎	(5 预设 + 8 强调色, localStorage 持久化)
☐ 生产构建部署	(254ms 构建, Nginx 静态资源 + /api/odoo 代理已启用)
☐ 两段式后端架构设计完成	(oweb → odoo-web-server → Odoo → PostgreSQL)
☐ API 代理链路验证通过	(Nginx → Odoo JSON-RPC 往返 < 50ms)
☐ odoo-web-server 搭建	(Rust workspace + axum 骨架)
☐ Session 代理	(Rust session 验证 + Cookie 透传)
☐ Odoo Bus → WebSocket 桥接	(实时通知)
☐ 路由守卫	(认证/权限拦截)
☐ 首个业务模块	(CRM/销售)

九、参考资源与优先级

开发过程中遵循以下参考优先级：

Odoo 19 CE 源码 > 第三方开源项目 > 其他通用参考

参考资料层级

优先级	来源	用途	方向
P0	~/EA/odoo (Odoo 19 CE 源码)	API 契约、Model 定义、JSON-RPC 协议、Session 机制、Bus 事件	全栈
P1	~/EA/l8-erp-react (Odoo React 前端)	React Odoo 组件模式、列表/表单/搜索、RPC 客户端、数据表	前端
P1	odoo-rust-mcp	Rust 侧 Odoo 客户端封装、request 模式、axum 集成	后端
P2	~/EA/Rustdoo (odoo-rust)	业务模型 Rust struct 参照、Session 管理、Web 服务器配置	后端
P3	~/EA/unicode (unicode-platform)	axum Router 组织、WebSocket 广播、静态文件 serve	全栈

9.1 P0 — Odoo 19 Community Edition 源码

路径: ~/EA/odoo

关键目录:

目录	内容	odoo-web-server 参考点
odoo/http.py	HTTP 请求处理、JSON-RPC 调度	Odoo JSON-RPC 协议规范
odoo/service/model.py	服务方法注册	common.authenticate 等标准服务
odoo/addons/web/controllers/	Web 客户端 API 端点	/web/session/* , /web/dataset/*
odoo/addons/base/models/	基础模型 (res.users, res.partner)	核心 Model 字段定义
odoo/orm/	ORM 引擎	模型字段类型、domain 语法、关系映射
odoo/addons/bus/	实时事件总线	Bus 轮询协议、事件格式

核心 API 端点 (Odoo 19 CE 标准路径):

端点	方法	说明
POST /jsonrpc	call	通用 JSON-RPC 入口
POST /web/session/authenticate	—	用户登录 (返回 session_id)
POST /web/session/get_session	—	获取当前会话信息
POST /web/session/destroy	—	销毁当前会话
POST /web/dataset/search_read	—	搜索并返回记录
POST /web/dataset/call_kw	—	通用 ORM 调用
POST /web/bus/poll	—	事件轮询
GET /web/content/*	—	附件/图片下载

9.2 P1 (前端) — l8-erp-react

路径: ~/EA/l8-erp-react

定位: Odoo ERP 的 React 前端 monorepo (pnpm + turborepo) , 包含完整的 Odoo 业务 UI 组件库。

规模: [packages/biz-ui](#) (业务组件) + [packages/odoo-rpc](#) (JSON-RPC 客户端) + [packages/oweb-core](#) (核心框架) + [apps/oweb](#) (应用)

核心模块及对 OdooSeek 的参考价值:

模块	文件	行数	参考点
----	----	----	-----

模块	文件	行数	参考点
Odoo JSON-RPC 客户端	packages/odoo-rpc/src/client.ts	~300	searchRead/search/write/create/unLink 方法签名、双端点 fallback、Session Cookie 管理
动态列表渲染	packages/biz-ui/components/views/list-preview.tsx	~400	Odoo 数据列表 + 虚拟滚动 (>80 行) + 行选择 + 排序
动态表单渲染	packages/biz-ui/components/views/form-preview.tsx	738	全部字段类型分发: char→input, text→textarea, boolean→checkbox, selection→select, many2one→关联, date→日期, image→图片
搜索面板	packages/biz-ui/components/search/	—	Odoo SearchView domain 构建、多条件过滤、分组聚合
统一查询 Hook	packages/biz-ui/hooks/use-preview-queries.ts	1844	构建 list/kanban/pivot/graph 视图查询的 master hook
URL 状态同步	apps/oweb/src/hooks/use-table-url-state.ts	~100	TanStack Table 状态 ↔ URL search params 双向同步
认证状态管理	apps/oweb/src/stores/auth-store.ts	~200	Zustand 实现 login/restore/logout, Cookie 管理
通用表格子组件	apps/oweb/src/components/data-table/	—	toolbar/pagination/faceted-filter/column-header/view-options
WebSocket 管理	apps/oweb/src/features/discuss/bus/	—	Odoo Bus WebSocket 连接、重连、事件分发

技术栈对比:

方面	l8-erp-react	OdooSeek (oweb)
UI 组件库	shadcn/ui (Radix)	纯 Tailwind CSS
HTTP 客户端	axios	fetch
状态管理	Zustand	TanStack Query + Context
路由	TanStack Router (v1 file-based)	TanStack Router (v1 code-based)
构建	Vite	Vite
包管理	pnpm + turborepo	Bun

可借鉴模式（即使技术栈不同）：

- FormPreview 的字段类型分发逻辑 — 可直接翻译为本项目的 OdooFormView
- ListPreview 的虚拟滚动 + 行选择 + 排序 — 可简化为本项目的 OdooListView
- search_read 的参数签名（domain/fields/limit/offset/order/context） — 与我们现有 SDK 对齐

- [DataTableToolbar](#) 的搜索 + 过滤 + 列可见性框架 — 可参考 UI 布局

不适用点: 依赖 shadcn/ui 生态（Radix primitives + cmdk + lucide-react），组件不能直接复用。

9.3 P1 (后端) — odoo-rust-mcp

路径: [~/EA/odoo-rust-mcp](#) ([GitHub](#))

定位: 为 Cursor/Claude Desktop 等 AI 助手提供 MCP 协议的工具服务器。

参考模块	具体内容
src/odoo/legacy_client.rs	reqwest::Client 构建 (cookie_store, timeout, user_agent)
src/odoo/legacy_client.rs	JSON-RPC call(service, method, args) 到 /jsonrpc
src/odoo/client.rs	Odoo 19+ HTTP 客户端模式 (Bearer token, headers)
src/odoo/config.rs	OdooInstanceConfig 多实例配置加载
Cargo.toml	依赖版本: reqwest 0.12 (json+cookies+rustls-tls), axum 0.8 (ws), tower-http 0.6 (cors+fs)

不适用点: MCP 的"工具语义"与 BFF 的"聚合 API 语义"是两种不同的抽象。

9.4 P2 — Rustdoo ([odoo-rust](#))

路径: [~/EA/Rustdoo](#)

定位: Odoo ERP 的完整 Rust 重写 (53 crates, ~186K LOC)。

参考模块	文件	行数	说明
Web 框架	odoo_web/src/server.rs	375	ServerConfig + 请求处理
Session 管理	odoo_web/src/sessions.rs	~300	SessionManager , SessionConfig
服务器入口	odoo_server/src/main.rs	188	axum Router 组装 + 优雅关闭
JSON-RPC	odoo_api/src/jsonrpc.rs	—	Odoo JSON-RPC 封装
CRM 模型	odoo_crm/src/models/crm_lead.rs	1368	线索/商机字段定义
会计分录	odoo_account/src/models/account_move_line.rs	881	凭证行 52+ 字段
ORM	odoo_orm/src/queries/	—	sqlx 直连 PostgreSQL

局限性: 20 个 crate 无法编译，适合作为"参考图书馆"查阅，不适合直接引入依赖。

9.5 P3 — unicode (unicode-platform)

路径: `~/EA/unicode/crates/unicode-platform`

定位: AI Agent 编码系统的 Web 后端，本项目脚手架的 `axum` 模式来源。

参考模块	行号	说明
<code>src/main.rs</code> Router	779-794	<code>Router::new()</code> + <code>CorsLayer::permissive()</code> + <code>ServeDir</code> fallback
<code>src/main.rs</code> WebSocket	—	<code>broadcast::channel</code> + <code>ws.on_upgrade</code> 模式
<code>src/main.rs</code> request	691	<code>Client::builder().user_agent().build()</code> 模式
<code>Cargo.toml</code>	—	workspace 依赖组织 + profile 配置

9.6 构建路线（基于参考优先级）

第一阶段：骨架

- └─ P3 参照 `unicode-platform/src/main.rs` → `axum` 启动 + Router + CORS + `ServeDir`
- └─ P0 参照 `~/EA/odoo/odoo/http.py` → Odoo JSON-RPC 协议规范
- └─ P1 参照 `odoo-rust-mcp/src/odoo/` → request client 构建模式
- └─ P1 参照 `odoo-rust-mcp Cargo.toml` → 依赖版本

第二阶段：Session 代理

- └─ P0 参照 `~/EA/odoo/addons/web/controllers/session.py` → `authenticate/get_session/destroy` API
- └─ P2 参照 `Rustdoo/odoo_web/src/sessions.rs` → `SessionConfig` 结构设计
- └─ P1 参照 `odoo-rust-mcp config` → Odoo 连接多实例管理

第三阶段：API 聚合

- └─ P0 参照 `~/EA/odoo/odoo/orm/` → Model 字段类型 + domain 语法
- └─ P0 参照 `~/EA/odoo/addons/web/controllers/dataset.py` → `search_read/call_kw`
- └─ P2 参照 `Rustdoo/odoo_api/src/jsonrpc.rs` → JSON-RPC 调用封装
- └─ P2 参照 `Rustdoo` 业务模型 struct → REST 响应格式设计

第四阶段：高性能扩展

- └─ P2 参照 `Rustdoo/odoo_orm/src/queries/` → `sqlx` 直连 PostgreSQL
- └─ P1 参照 `odoo-rust-mcp` 元数据缓存 → `moka/dashmap` 内存缓存

十、命名释义

名称	含义
OdooSeek	探寻 Odoo 技术边界的旅程 (Seek = 追寻、探索)
oweb	Odoo Web 的重新诠释，既是 <code>odoo + web</code> 也是 <code>our web</code>
odoo-web-server	Rust 构建的中间层 BFF，连接 oweb 与 Odoo 的智能网关

"我们不是在逃离 Odoo，而是在拓宽它的边界。"
— OdooSeek 团队

文档版本: 1.7

更新日期: 2026-05-28

维护团队: OdooSeek